



Artificial Neural Network

Dr. Prasenjit Dey,
Assistant Professor,
NIT Rourkela,
Odisha, India

Content

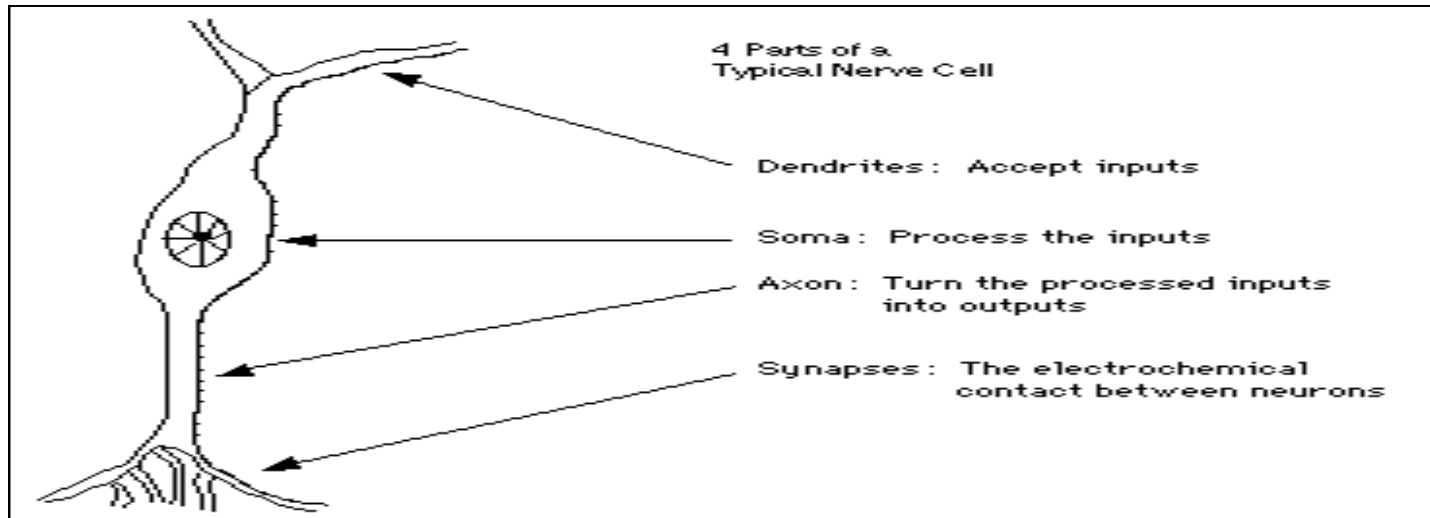
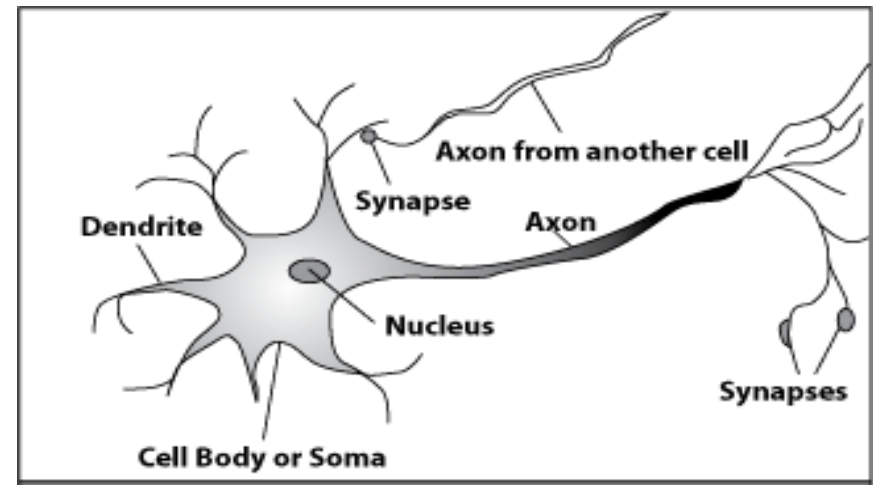
- How do Brain Neural Network (BNN) works?
- Artificial Neural network (ANN)
- Difference between ANN and BNN
- McCulloch-Pitts Neuron Model
- Activation Function
- Perceptron
- Multi Layer feed forward NN
- Backpropagation
- NN Design issues
- Learning Rules

History

- late-1800's - Neural Networks appear as an analogy to biological systems
- 1960's and 70's – Simple neural networks appear
 - The perceptron is not effective for non linear classification.
- 1986 – Backpropagation algorithm appears
 - Neural Networks have a resurgence in popularity
 - More computationally expensive

How do our brains work?

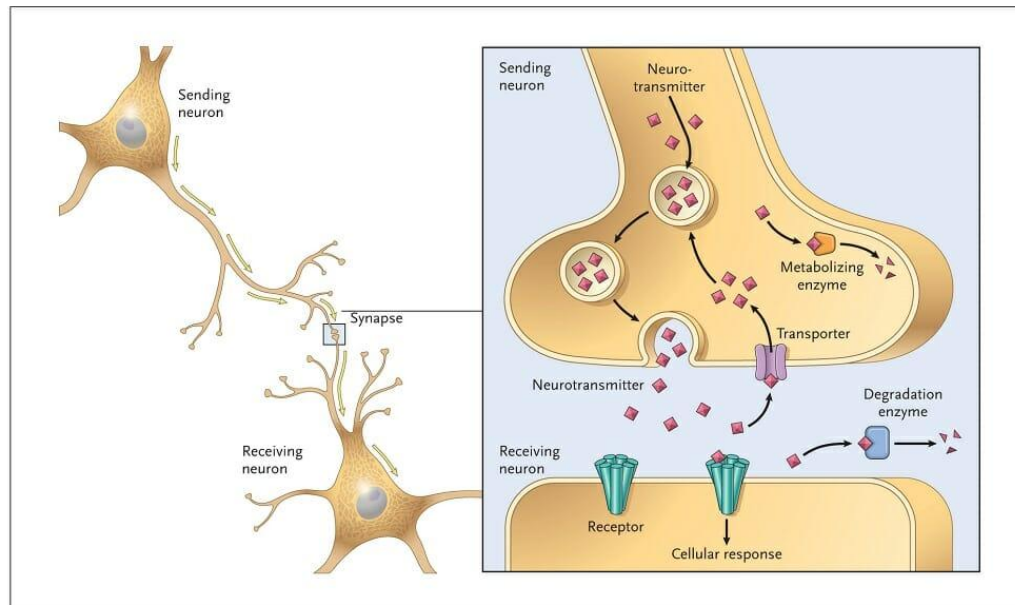
- The Brain is a massively parallel information processing system.
- Our brains are a huge network of processing elements. A typical brain contains a network of nearly 10 billion neurons.



How do our brains work?

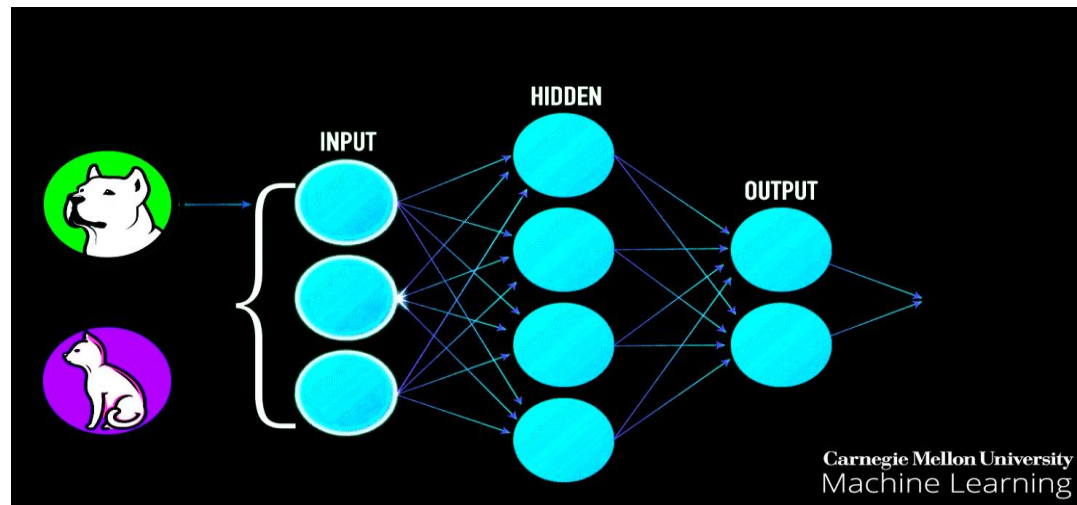
- Neurons connect through approximately 10,000 synapses.
- Neurons receive input from other neurons and combine it.
- When input exceeds a critical level, a spike is discharged, an electrical pulse.
- The axon endings nearly touches the next neuron's dendrites.
- Neurotransmitters transmit electrical signals between neurons.
- The strength of the signal depends on the amount of neurotransmitter available.

Generic Neurotransmitter System



Artificial neural network(ANN)

- An ANN is specified by:
 - **neuron model(perceptron)**: the information processing unit of the NN,
 - **an architecture**: a set of neurons and links connecting neurons. Each link has a weight,
 - **a learning algorithm**: used for training the NN by modifying the weights in order to model a particular learning task correctly on the training examples.
- The aim is to obtain a NN that is trained and generalizes well.
It should behaves correctly on new instances of the learning task.



Artificial neural network

- ANN is a machine learning approach that models human brain and consists of a number of artificial neurons.
- Neuron in ANNs tend to have fewer connections than biological neurons.
- Each neuron in ANN receives a number of inputs.
- An activation function is applied to these inputs which results in activation level of neuron (output value of the neuron).
- Knowledge about the learning task is given in the form of examples called training examples.

BNN VS ANN

Characteristics	BNN	ANN
Speed	Processes information at a slower rate. Response time is measured in milliseconds.	Information is processed at a faster rate. The response time is measured in nanoseconds.
Processing	Massively parallel processing.	Serial processing.
Size & Complexity	An extremely intricate and dense network of linked neurons of the order of 10 ¹¹ neurons and 10 ¹⁵ interconnections.	Size and complexity are reduced. It is incapable of performing sophisticated pattern recognition tasks.
Storage	An extremely intricate and dense network of linked neurons with 10 ¹⁵ interconnections, including neurons on the order of 10 ¹¹ .	The term "replaceable information storage" refers to the practice of replacing fresh data with old data.
Fault Tolerance	The fact that information storage is flexible means that new information may be added by altering the connectivity strengths without deleting existing information.	Intolerant of faults. In the event of a system failure, corrupt data cannot be recovered.
Control Mechanism	There is no unique control mechanism outside of the computational task	Controlling computer activity is handled by a control unit.

Neural Networks

- **Neural Network** learns by adjusting the weights so as to be able to correctly recognize the training data and hence, after testing phase, to recognize unknown data.
- It needs long time for training.
- It has a high tolerance to noisy and incomplete data

Input: Classification data

It contains classification attribute

Data is divided, as in any classification problem.

[Training data and Testing data]

All data must be normalized.

(i.e. all values of attributes in the database are changed to contain values in the interval $[0,1]$)

Basic normalization techniques

- Max-Min normalization
- Decimal Scaling normalization

Max-Min normalization

$$v' = \frac{v - \min A}{\max A - \min A} (\text{new_max } A - \text{new_min } A) + \text{new_min } A$$

v is the **original value of the attribute (feature)** before normalization.

v' is the **new normalized value** after applying the normalization formula.

$\min(A)$ means the **minimum value of attribute A in the dataset**.

$\max(A)$ means the **maximum value of attribute A in the dataset**.

Max-Min normalization

Example: We want to normalize data to range of the interval [0, 1].

We put: new_maxA= 1, new_minA= 0.

Let us use, max A was 100 and min A was 20 (That means maximum and minimum values for the attribute).

Now, if $v = 40$ (If for this particular pattern , attribute value is 40), v' will be calculated as , $v' = (40-20) \times (1-0) / (100-20) + 0$

$$\Rightarrow v' = 20 \times 1/80$$

$$\Rightarrow v' = 0.25$$

Decimal Scaling normalization

Normalization by decimal scaling normalizes by moving the decimal point of values of attribute A.

$$v' = \frac{v}{10^j}$$

Here j is the smallest integer such that $\max |v'| < 1$.

Example :

A – values range from -986 to +917. $\max |v| = 986$.

Since $986/10^2 = 9.86 (> 1)$ **is not valid**

Next $986/10^3 = 0.986 (< 1)$ **is valid**

Therefore, $v' = 0.986$ and $j = 3$

When to consider neural networks

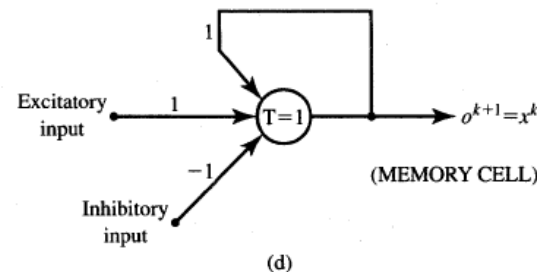
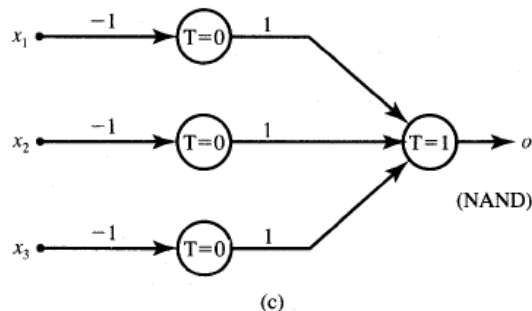
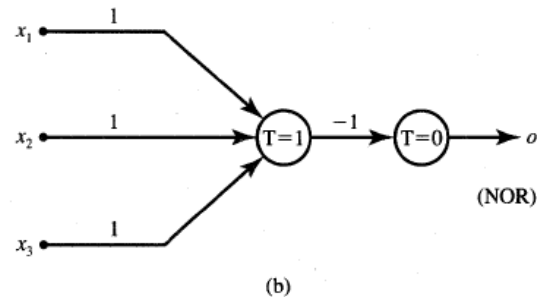
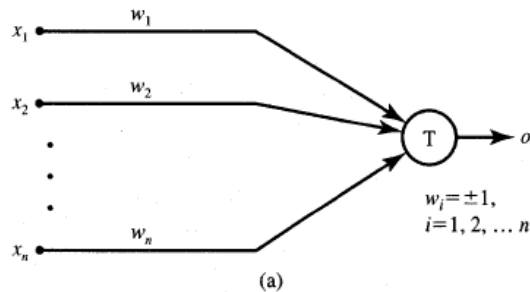
- Input is high-dimensional discrete or raw-valued
- Output is discrete or real-valued
- Output is a vector of values
- Possibly noisy data
- Form of target function is unknown
- Human readability of the result is not important

Examples:

- Speech phoneme recognition
- Image classification
- Financial prediction

McCulloch-Pitts Neuron Model

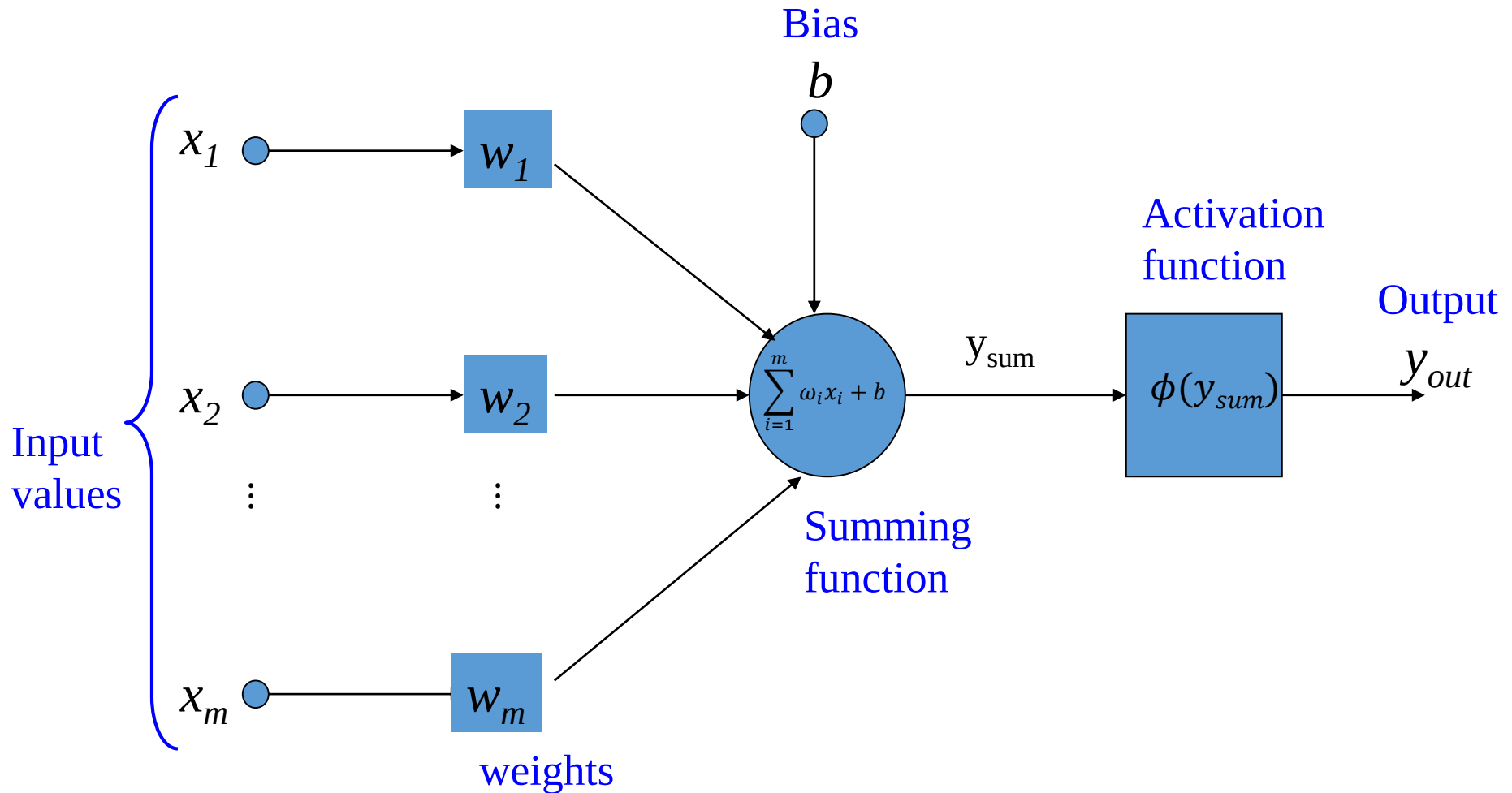
$$o^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{if } \sum_{i=1}^n w_i x_i^k < T \end{cases}$$



Features of McCulloch-Pitts model

- Allows binary 0,1 states only
- Operates under a discrete-time assumption
- Weights and the neurons' thresholds are fixed in the model and no interaction among network neurons

The Neuron Diagram

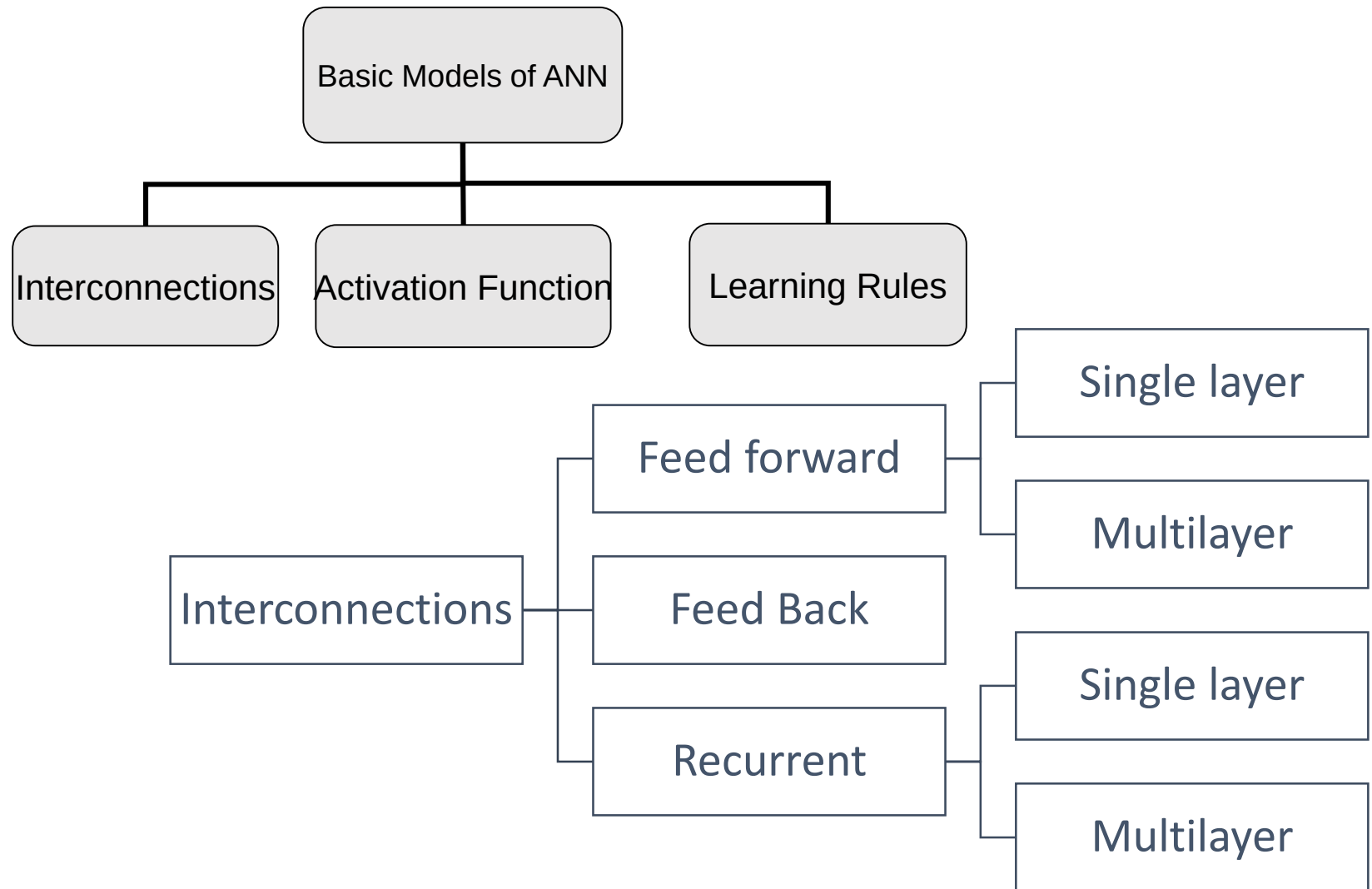


Bias of a Neuron

- The bias b has the effect of shifts the weighted sum $w_i x_i$
- The bias is an external parameter of the neuron. It can be modeled by adding an extra input.
- v is called the induced field of the neuron
- Bias allows the model to fit data even when input is zero.

$$v = \sum_{i=1}^m w_i x_i + b$$

Classification of Artificial Neural Network Models

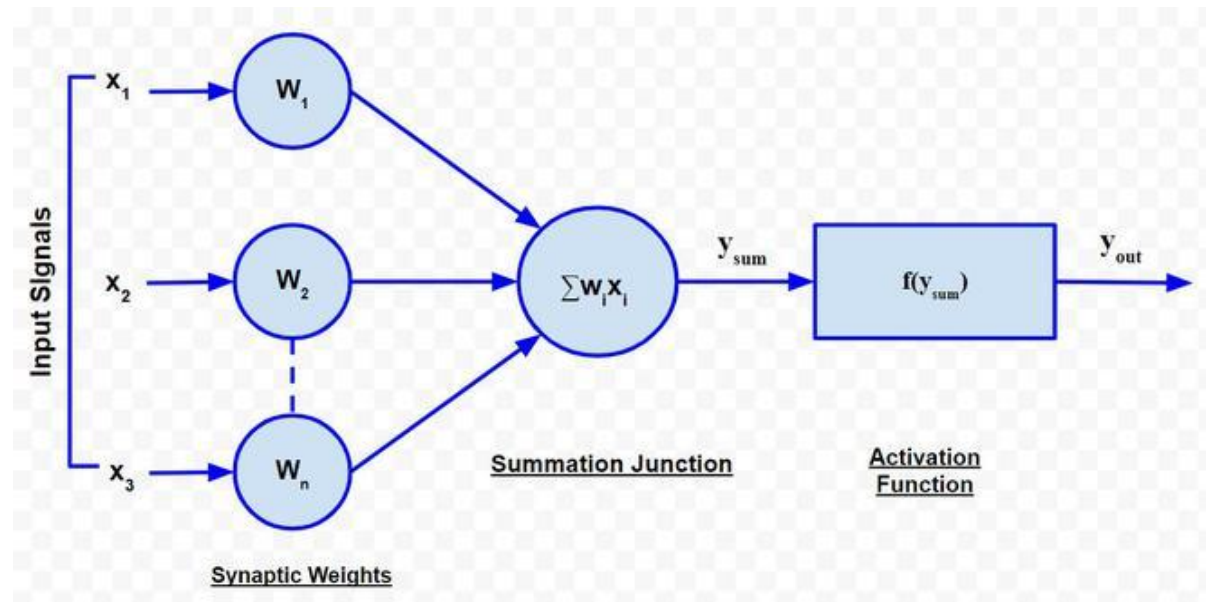


Activation Function

- The activation function decides whether a neuron should be activated or not by calculating the weighted sum and further adding bias to it. The purpose of the activation function is to introduce non-linearity into the output of a neuron.

Different types of activation function are:

- step function
- ramp function
- sigmoid function
- Gaussian function
- ReLu
- Leaky ReLu



Step Function

$$\varphi(v) = \begin{cases} a & \text{if } v < c \\ b & \text{if } v > c \end{cases}$$

Characteristics of the Step Activation Function:

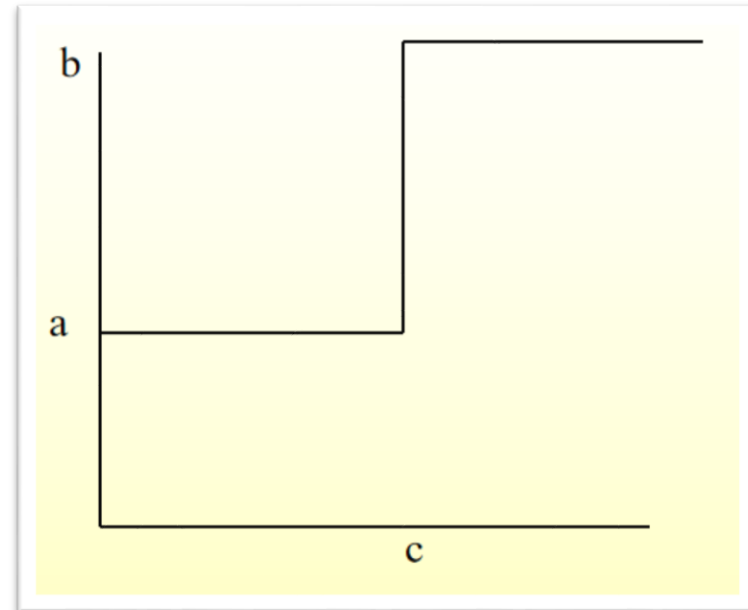
- Non-linearity
- Binary Output
- Threshold-Based

Advantages:

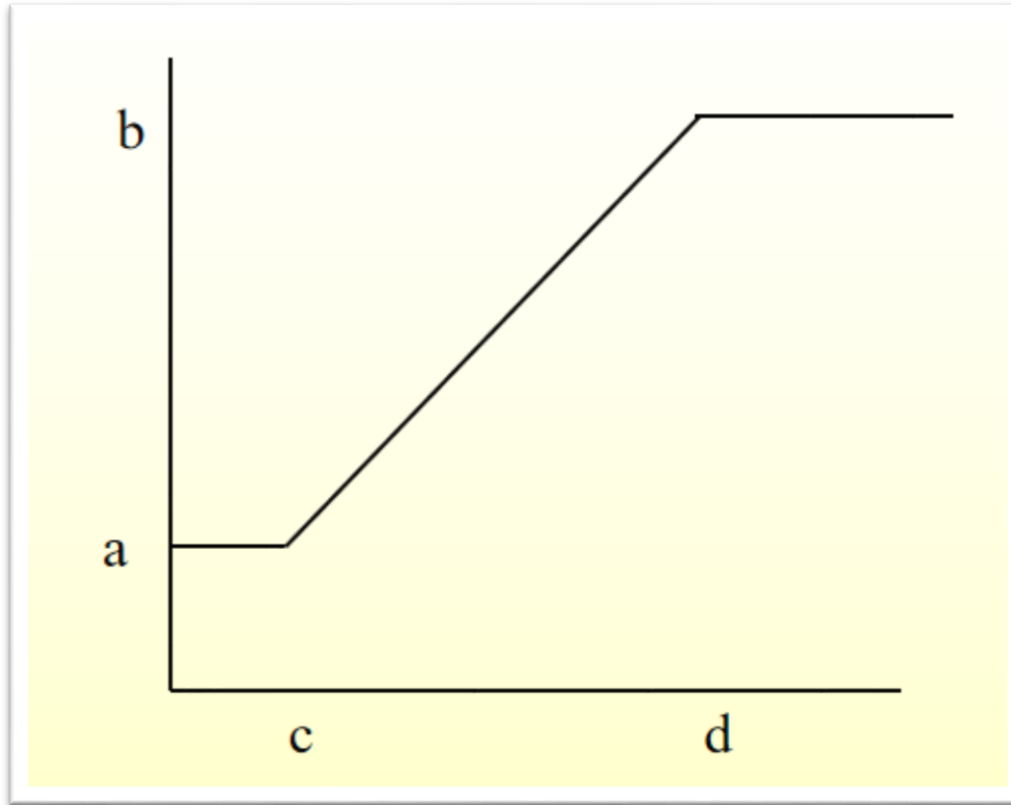
- Simplicity
- Clear Decision Boundaries

Disadvantages:

- Non-Differentiability: This makes it unsuitable for use with gradient-based optimization methods like backpropagation.
- Learning Challenges: Because of its zero gradient, the step function doesn't provide useful information for updating the weights during training



Ramp Function



$$\varphi(v) = \begin{cases} a & \text{if } v < c \\ b & \text{if } v > d \\ a + ((v - c)(b - a) / (d - c)) & \text{otherwise} \end{cases}$$

Sigmoid function

$$\varphi(v) = \frac{1}{1 + e^{-Av}}$$

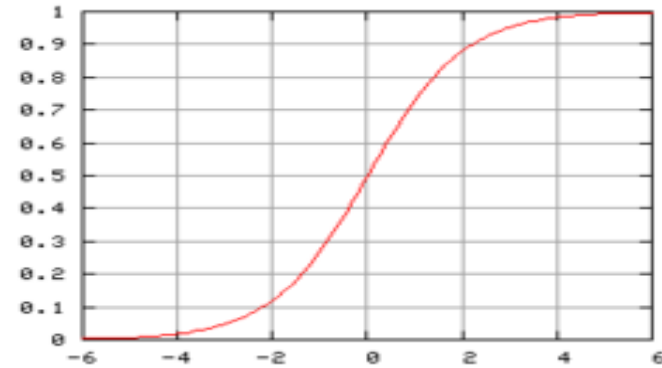
- A is a constant called the slope parameter (gain).
- It controls **how steep the sigmoid curve is**.
- In most neural network model $A = 1$

Advantages:

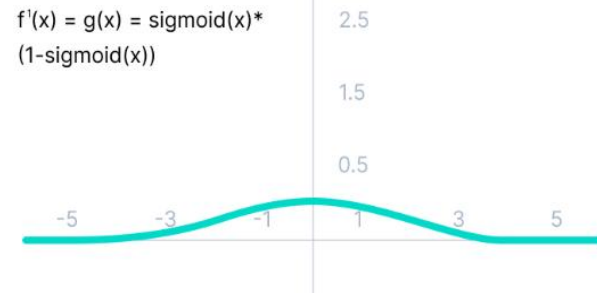
- **Smooth Gradient during optimization process.**
- **Probabilistic Interpretation:** Outputs are in the range (0, 1), which can be interpreted as probabilities, making the sigmoid function useful for binary classification tasks.

Disadvantages:

- **Vanishing Gradient Problem:** For very large or very small input values, the gradient of the sigmoid function becomes very small
- **Output Saturation:** When the input is very high or very low, the output of the sigmoid function tends to saturate at 1 or 0, respectively.



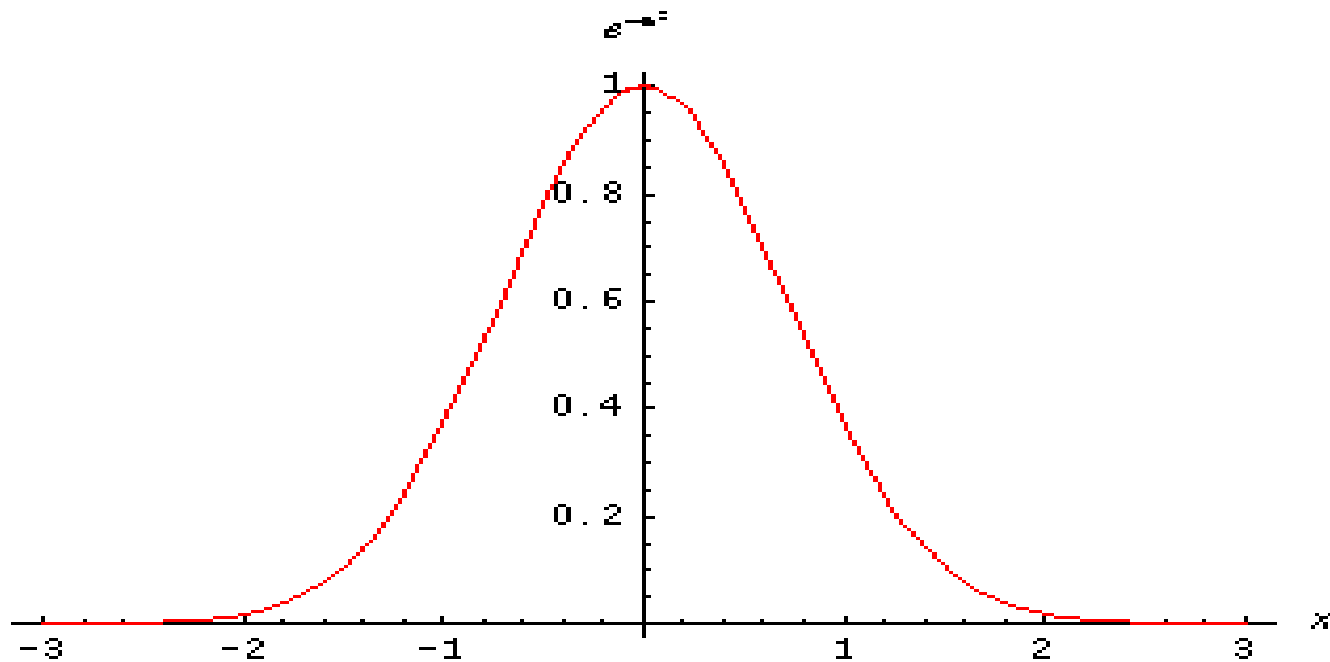
$$f'(x) = g(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$$



Gaussian function

- The probability function of the normal distribution. Sometimes also called the frequency curve.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$$



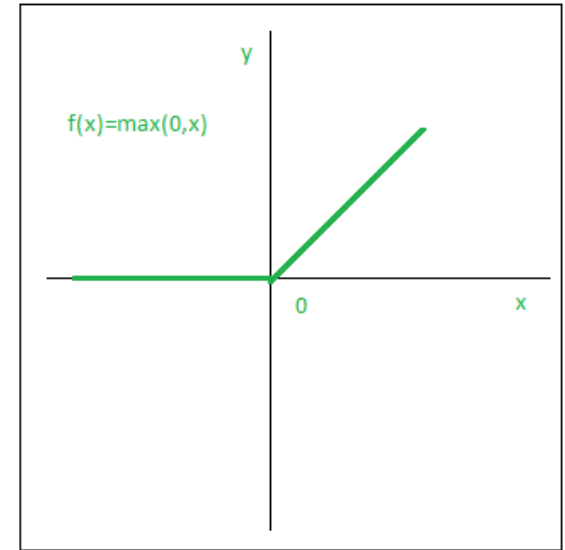
ReLU function

The ReLU function is the Rectified linear unit.

$$f(x) = \max(0, x)$$

The main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time. ?

If you look at the ReLU function if the input is negative it will convert it to zero and the neuron does not get activated



Dead ReLU Neuron

- Suppose $net_1 < 0$
- $a_1 = f(net_1) = \max(0, net_1) = 0$
- $\frac{\partial a_1}{\partial net_1} = f'(net_1) = 0$
- No weight update
- No learning
- h_1 is a dead ReLU unit which can not recover

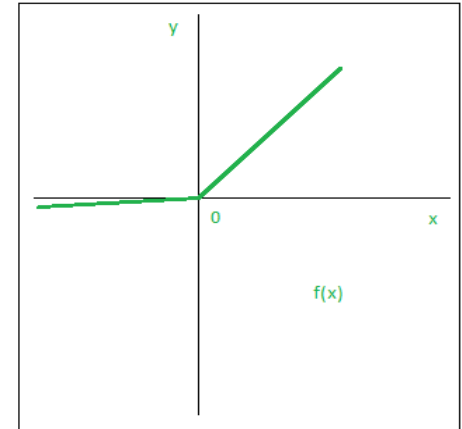
Leaky ReLu

Instead of defining the Relu function as 0 for x less than 0, we define it as a small linear component of x .

It can be defined as:

$$f(x) = ax, x < 0$$

$$f(x) = x, \text{ otherwise}$$



ReLU is non-saturating

- For large $\sum w_i x_i > 0$, ReLU's output will be large
- Gradient will be large value

$$w_i = w_i - \alpha \frac{\partial L}{\partial w_i}$$

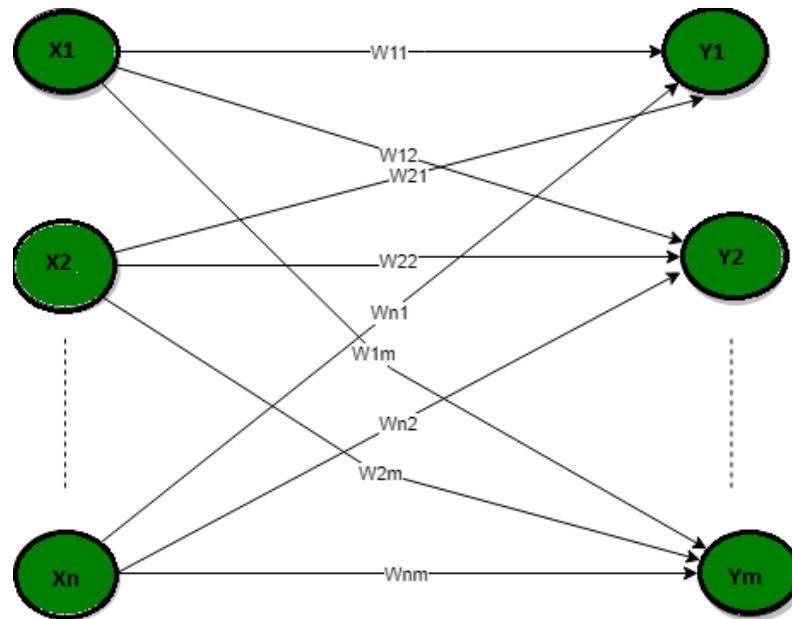
Exploding gradient problem

- So updates will be very large and gradient descent may be unstable

Network Architectures

- Three different classes of network architectures
 - single-layer feed-forward
 - multi-layer feed-forward
 - recurrent
- The **architecture** of a neural network is linked with the learning algorithm used to train

Single Layer Feed-forward

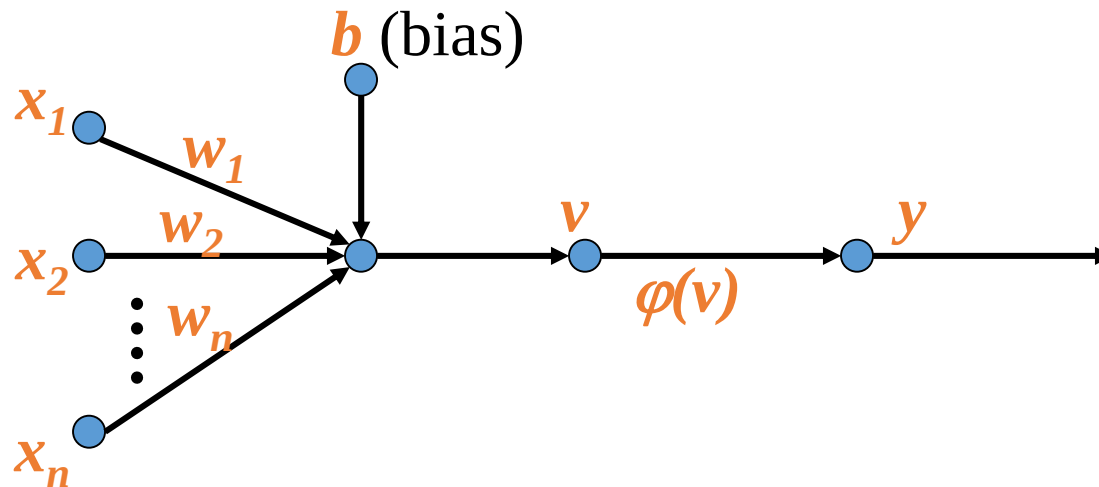


Perceptron: Neuron Model

(Special form of single layer feed forward)

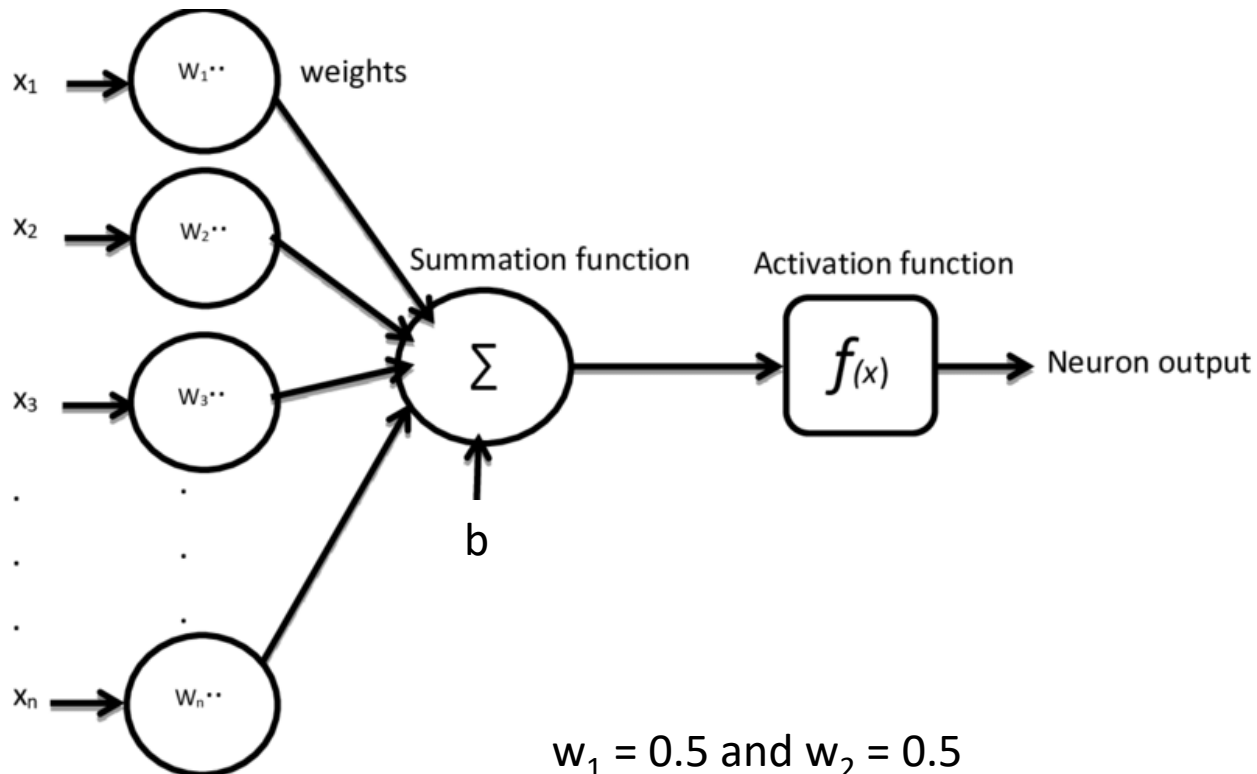
- The perceptron was first proposed by Rosenblatt (1958) is a simple neuron that is used to classify its input into one of two categories.
- A perceptron uses a **step function** that returns +1 if weighted sum of its input ≥ 0 and -1 otherwise

$$\phi(v) = \begin{cases} +1 & \text{if } v \geq 0 \\ -1 & \text{if } v < 0 \end{cases}$$



Perceptron for Classification

- The perceptron is used for binary classification.
- First train a perceptron for a classification task.
 - Find suitable weights in such a way that the training examples are correctly classified.
 - Geometrically try to find a hyper-plane that separates the examples of the two classes.
- The perceptron can only model linearly separable classes.
- When the two classes are not linearly separable, it may be desirable to obtain a linear separator that minimizes the mean squared error.
- Given training examples of classes C_1 , C_2 train the perceptron in such a way that :
 - *If the output of the perceptron is +1 then the input is assigned to class C_1*
 - *If the output is -1 then the input is assigned to C_2*



$$w_1 = 0.5 \text{ and } w_2 = 0.5$$

$$\text{Let, } x_1 = 0.3 \text{ and } x_2 = 0.8,$$

Then, weighted sum is:

$$w_1x_1 + w_2x_2 = 0.5 * 0.3 + 0.5 * 0.8 = 0.55$$

Learning Process for Perceptron

- Initially assign random weights to inputs $[-0.5, 0.5]$
- Training data is presented to perceptron and its output is observed.
- If output is incorrect, the weights are adjusted accordingly using following formula.

$$w_i \leftarrow w_i + (\eta * e * x_i),$$

where e is the error and η ($0 < \eta < 1$) is the learning rate

- $$e = \begin{cases} 0 & \text{if output is correct} \\ +ve & \text{if output is too low} \\ -ve & \text{if output is too high} \end{cases}$$
- Repeat until all errors are minimized
- One complete pass is called an epoch

Example: Perceptron to learn OR function

- Initialize weights: $w_1=0.6$, $w_2=0.6$

- Step function: $y = \begin{cases} 1, & \text{if net} \geq 1 \\ 0, & \text{if net} < 1 \end{cases}$

- Training:

1. $x_1 = 0, x_2 = 0$

Net = 0 $\rightarrow$ $y=0$, $d=0$ (correct)

2. $x_1 = 0, x_2 = 1$

Net = 0.6 $\rightarrow$ $y=0$, $d=1$ (wrong)

Update weights(for incorrect case):

$$w \leftarrow w + \eta ex$$

Take $\eta = 0.5$, $e = (d - y) = (1 - 0) = 1$

$$w_1 = 0.6 + (0.5 * 1 * 0) = 0.6$$

$$w_2 = 0.6 + (0.5 * 1 * 1) = 1.1$$

x_1	x_2	w_1	w_2	OR(d)	y
0	0	0.6	0.6	0	0
0	1	0.6	0.6	1	0($w_1 \rightarrow 0.6, w_2 \rightarrow 1.1$)
1	0	0.6	1.1	1	0($w_1 \rightarrow 1.1, w_2 \rightarrow 1.1$)
1	1	1.1	1.1	1	1

3. $x_1 = 1, x_2 = 0$ (with update weights)

Net = 0.6 $\rightarrow$ $y=0$, $d=1$ (wrong)

Update weights(for incorrect case):

Take $\eta = 0.5$, $e = (d - y) = (1 - 0) = 1$

$$w_1 = 0.6 + (0.5 * 1 * 1) = 1.1$$

$$w_2 = 1.1 + (0.5 * 1 * 0) = 1.1$$

4. $x_1 = 1, x_2 = 1$ (with update weights)

Net = 2.2 $\rightarrow$ $y=1$, $d=1$ (correct)

Example: Perceptron to learn OR function

- Initialize weights: $w_1 = -0.2$, $w_2 = 0.4$

- Step function: $y = \begin{cases} 1, & \text{if net} > 0 \\ 0, & \text{if net} \leq 0 \end{cases}$

- Training:

1. $x_1 = 0, x_2 = 0$

Net = 0 $\rightarrow$ $y=0$, $d=0$ (correct)

2. $x_1 = 0, x_2 = 1$

Net = 0.4 $\rightarrow$ $y=1$, $d=1$ (correct)

3. $x_1 = 1, x_2 = 0$

Net = -0.2 $\rightarrow$ $y=0$, $d=1$ (wrong)

Update weights(for incorrect case):

$$w \leftarrow w + \eta ex$$

x_1	x_2	w_1	w_2	OR(d)	y
0	0	-0.2	0.4	0	0
0	1	-0.2	0.4	1	1
1	0	-0.2	0.4	1	0($w_1 \rightarrow 0, w_2 \rightarrow 0.4$)
1	1	0	0.4	1	1

Take $\eta = 0.2$, $e = (d - y) = (1 - 0) = 1$

$$w_1 = -0.2 + (0.2 * 1 * 1) = 0$$

$$w_2 = 0.4 + (0.2 * 0 * 1) = 0.4$$

4. $x_1 = 1, x_2 = 1$ (with update weights)

Net = 0.4 $\rightarrow$ $y=1$, $d=1$ (correct)

Update weights: $w_1 = 0$ and $w_2 = 0.4$

Repeat process until all outputs are correct

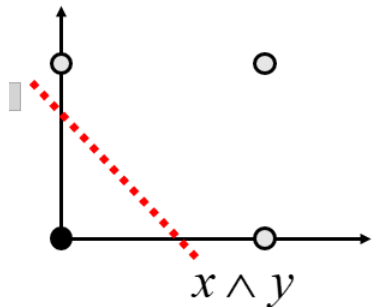
Perceptron: Limitations

- The perceptron can only model linearly separable functions.
- Boolean functions given below are linearly separable:
 - AND
 - OR
 - COMPLEMENT
- It cannot model XOR function as it is non linearly separable.
 - When the two classes are not linearly separable, it may be desirable to obtain a linear separator that minimizes the mean squared error.

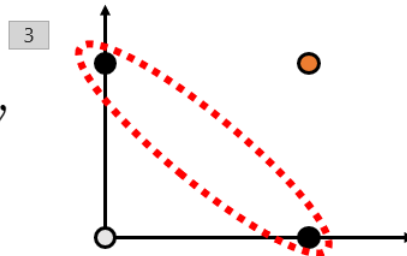
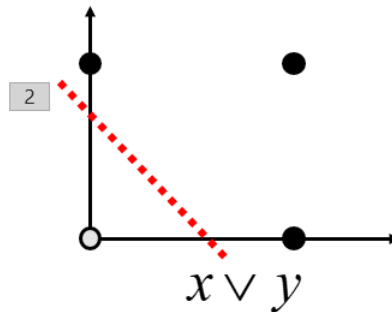
Linearly Separable

We say a two-dimensional dataset is linearly separable if we can separate the positive from the negative objects with a straight line.

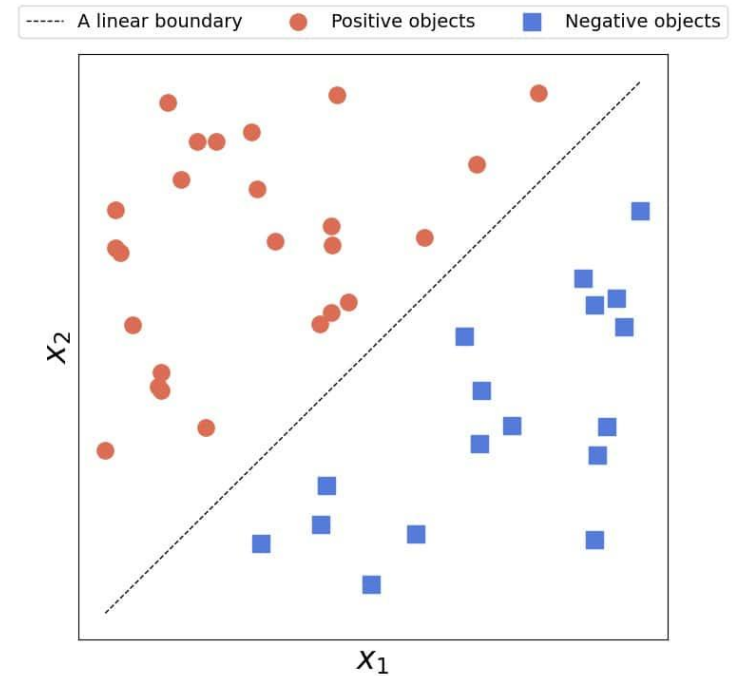
□ Linear Separable:



□ Linear inseparable:



□ Solution?



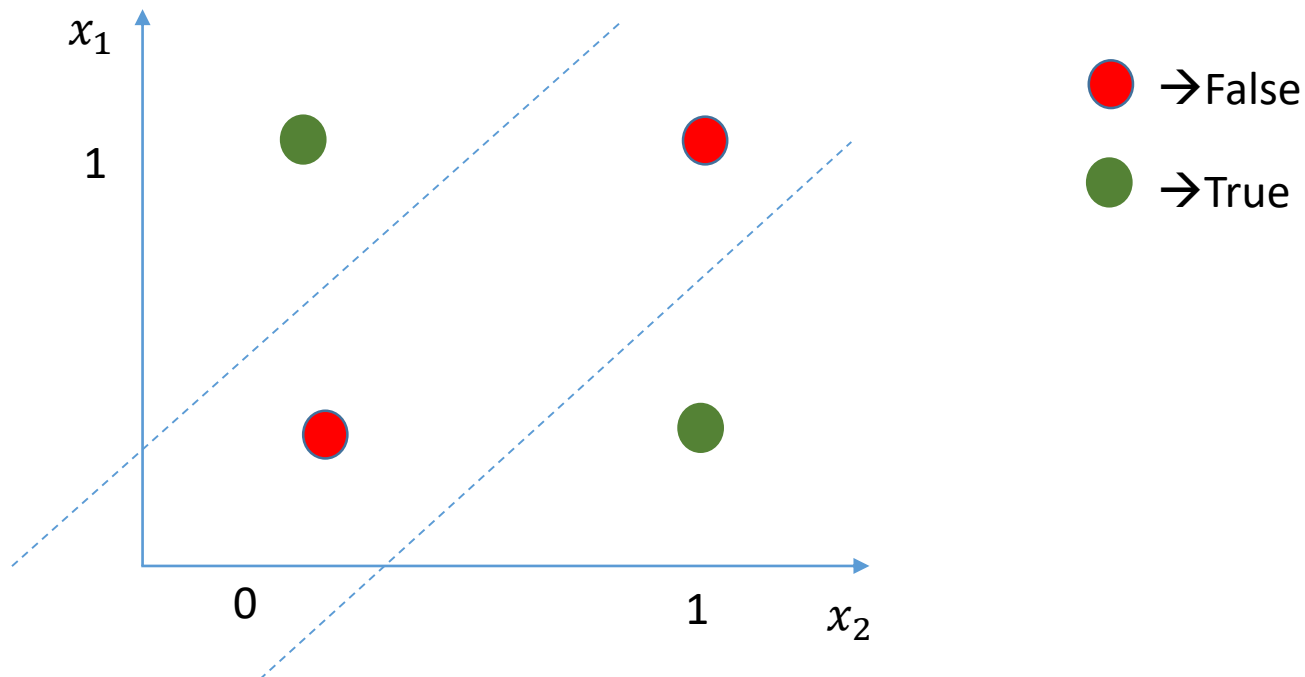
XOR – Non linearly separable function

- A typical example of non-linearly separable function is the XOR that computes the logical **exclusive or**.
- This function takes two input arguments with values in $\{0,1\}$ and returns one output in $\{0,1\}$,
- Here 0 and 1 are encoding of the truth values **false** and **true**,
- The output is **true** if and only if the two inputs have different truth values.

Input		Output
X_1	X_2	$X_1 \text{ XOR } X_2$
0	0	0
0	1	1
1	0	1
1	1	0

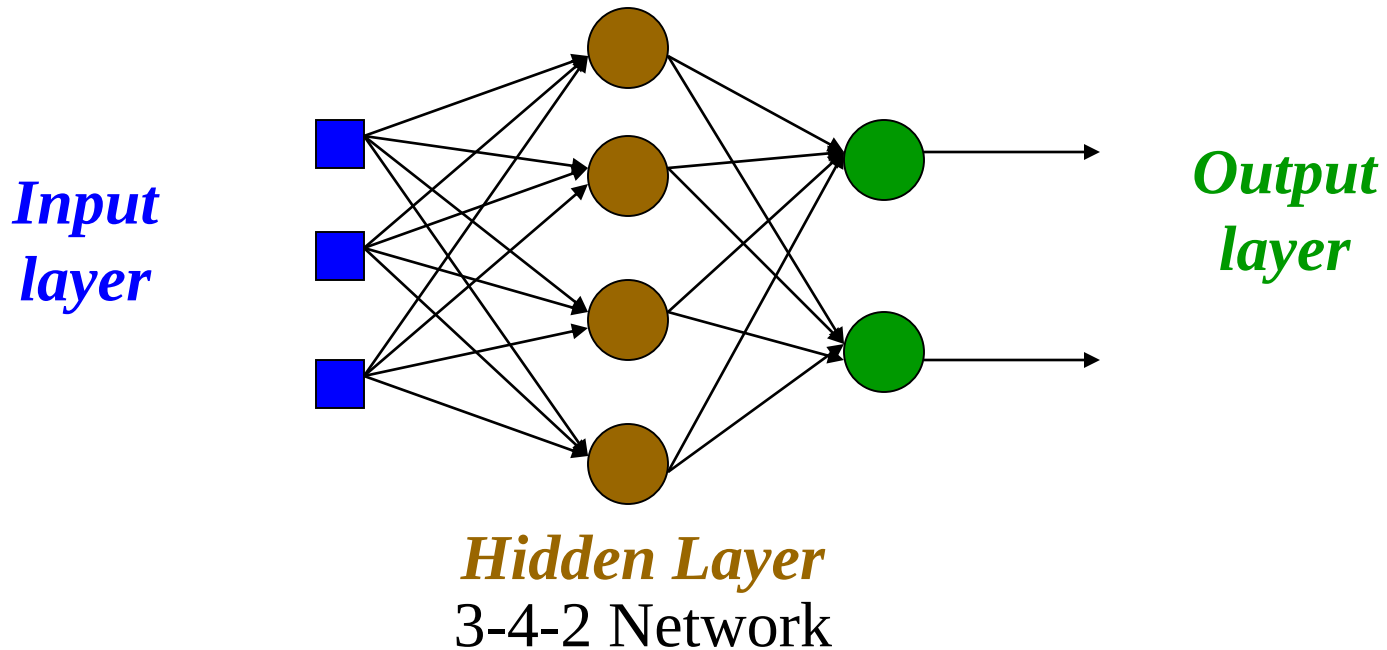
XOR – Non linearly separable function

- XOR is non linearly separable function which can not be modeled by perceptron.
- For such functions we have to use multi layer feed-forward network.
- These two classes (true and false) cannot be separated using a line. Hence XOR is non linearly separable.



Multi layer feed-forward NN (FFNN)

- FFNN is a more general network architecture, where there are hidden layers between input and output layers.
- Hidden nodes do not directly receive inputs nor send outputs to the external environment.
- They can handle non-linearly separable learning tasks.



Multi layer feed-forward NN (FFNN)

A network containing one hidden layer is called a **three-layer** neural network.

The network is feed-forward in that none of the weights cycles back to an input unit or to an output unit of a previous layer.

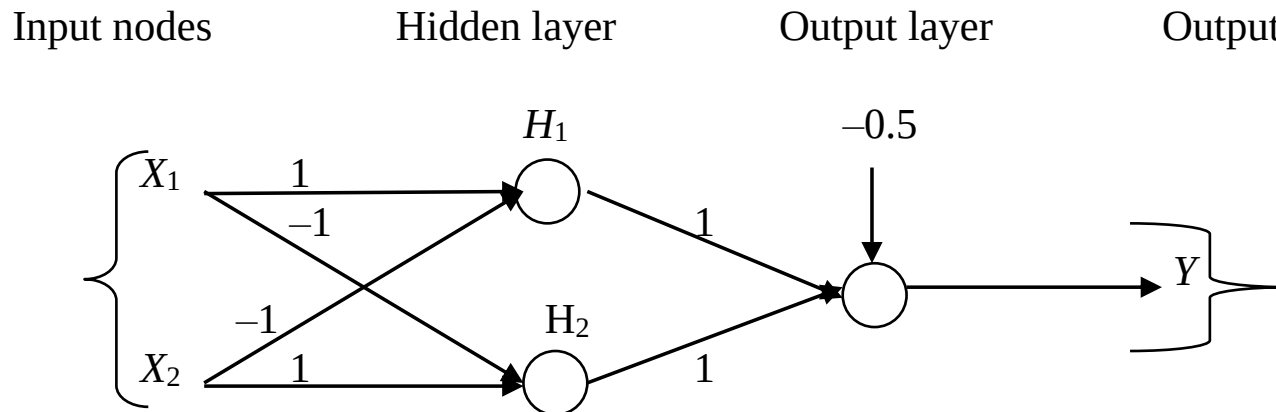
Input layer – The first layer that receives raw data and passes it forward without computation. The number of neurons equals the number of input features.

Hidden layer – the number of nodes in the hidden layer and the number of hidden layers depends on implementation.

Output layer – corresponds to the class attribute. There are as many nodes as classes (values of the class attribute).

FFNN for XOR

- The ANN for XOR has two hidden nodes that realizes this non-linear separation and uses the sign (step) activation function.
- Arrows from input nodes to two hidden nodes indicate the directions of the weight vectors $(1,-1)$ and $(-1,1)$.
- The output node is used to combine the outputs of the two hidden nodes.



FFNN for XOR

Inputs		Output of Hidden Nodes		Output Node	X_1 XOR X_2
X_1	X_2	H_1	H_2		
0	0	0	0	0.5 → 0	0
0	1	-1 → 0	1	0.5 → 1	1
1	0	1	-1 → 0	0.5 → 1	1
1	1	0	0	0.5 → 0	0

Since we are representing two states by 0 (false) and 1 (true), we will map negative outputs (-1 , -0.5) of hidden and output layers to 0 and positive output (0.5) to 1.

FFNN NEURON MODEL

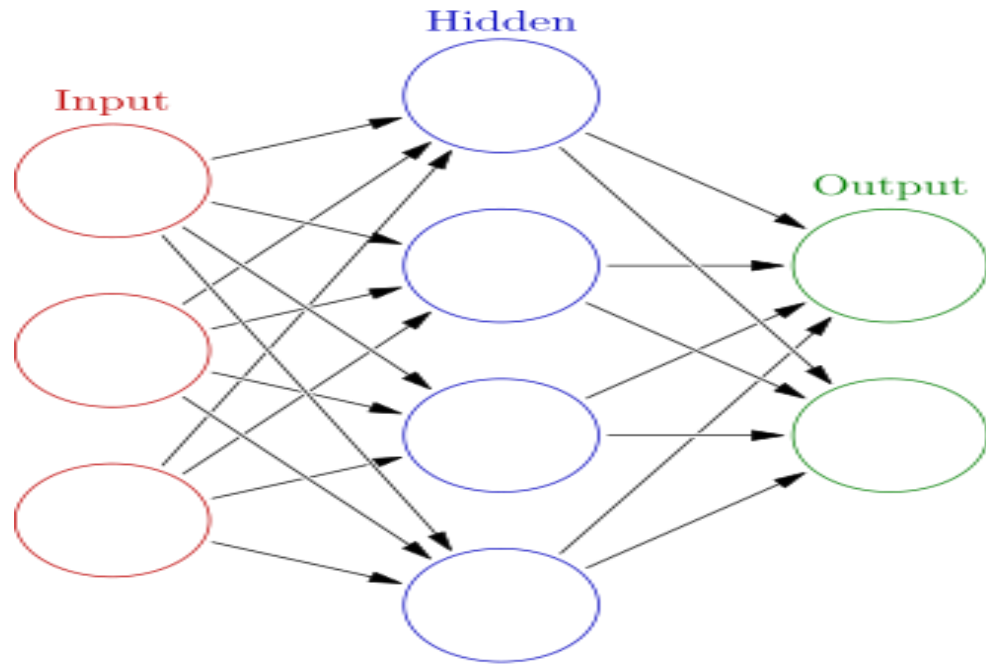
- The classical learning algorithm of FFNN is based on the gradient descent method.
- Activation function used in FFNN are continuous functions of the weights, differentiable everywhere.
- The activation function for node i may be defined as a simple form of the **sigmoid function** in the following manner:

$$\varphi(v) = \frac{1}{1 + e^{-v_j}}$$

where, $v_j = \sum_{i=1}^n w_{ij} \cdot x_i + b_j$, is the net input to the hidden node j , & x_i is the input of the i^{th} Input layer, and b is threshold for node j

Training Algorithm: Backpropagation

- Backpropagation is an extension of the perceptron learning rule used for training multi-layer neural networks.
- It searches for weight values that minimize the total error of the network over the set of training examples (training set).
- Backpropagation consists of the repeated application of the following two passes:
 - **Forward pass**
 - **Backward pass**



Contd..

- Consider a network of three layers.
- Let, i represents nodes in input layer, j represents nodes in hidden layer and k represents nodes in output layer.
- Let, w_{ij} refers to weight between a node i in input layer and node j in hidden layer.
- Then, the output y_j of node j

$$y_j = \frac{1}{1+e^{-v_j}}$$

Total Mean Squared Error

- The error of output neuron k after the activation of the network on the n^{th} training example $(x(n), d(n))$ is:

$$e_k(n) = d_k(n) - y_k(n)$$

- The network error is the sum of the squared errors of the output neurons:

$$E(n) = \frac{1}{2} \sum e_k^2(n)$$

- The total mean squared error is the average of the network errors of the training examples.

$$E_{AV} = \frac{1}{N} \sum_{n=1}^N E(n)$$

Gradient Descent and Delta Rule

- **Idea** – use gradient descent to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples,
- Assume transfer function (φ) is **identity** function (not Signum function) – [note that identity function (a linear unit) is used just for explanation]

$$\begin{aligned}v &= \mathbf{w}^t \mathbf{x} \\ \varphi(v) &= v\end{aligned}$$

Let's learn w_i 's that minimize the square error

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Where D is set of training examples

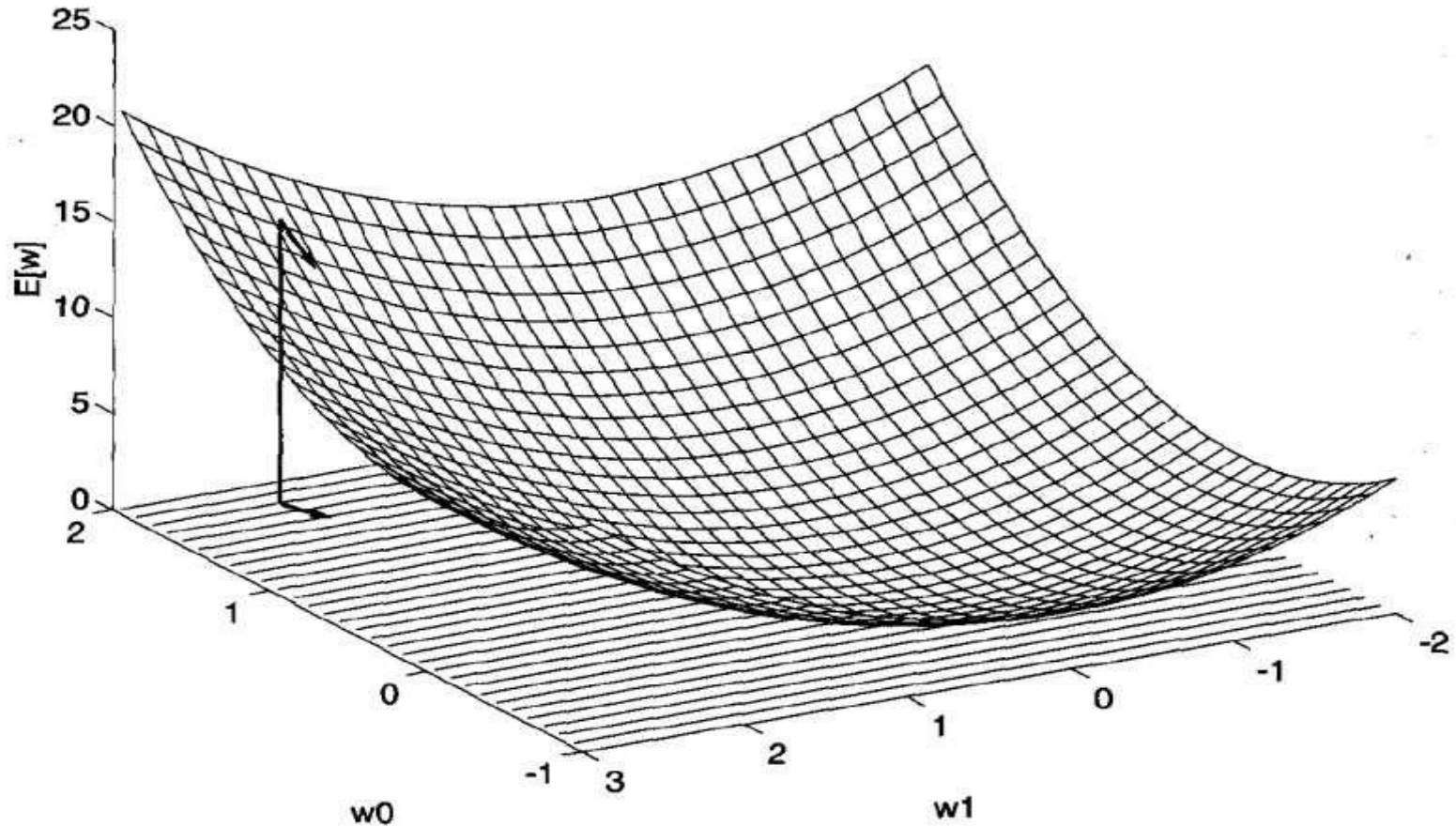
Gradient Descent and Delta Rule

Training Rule

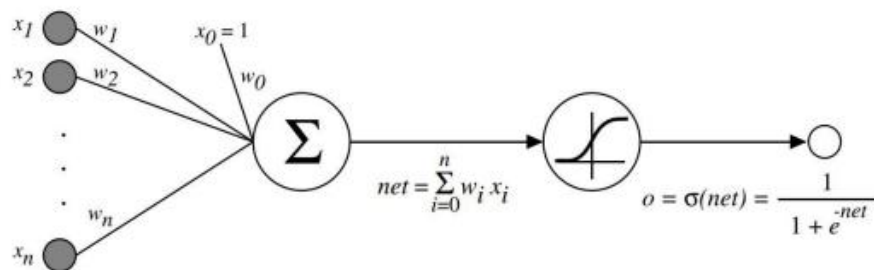
- $w_i = w_i + \Delta w_i$
- $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$
- $\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{i,d}$

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d 2 (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_d (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \mathbf{w} \cdot \mathbf{x}_d) \\ \frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d) (-x_{i,d})\end{aligned}$$

Visualizing The Hypothesis Space



Sigmoid Unit



sigmoid function:

$$\sigma(x) = \frac{1}{(1 + e^{-x})}$$

$$\sigma(x) = (1 + e^{-x})^{-1}$$

Differentiate using chain rule

$$\frac{\partial \sigma(x)}{\partial x} = -1(1 + e^{-x})^{-2} \frac{\partial}{\partial x} (1 + e^{-x})$$

$$= -1(1 + e^{-x})^{-2} (-e^{-x})$$

$$= \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$\sigma(x) = \frac{1}{(1 + e^{-x})}$$

$$1 - \sigma(x) = \frac{e^{-x}}{(1 + e^{-x})}$$

After multiplication

$$\sigma(x)(1 - \sigma(x))$$

$$= \left(\frac{1}{(1 + e^{-x})} \right) \left(\frac{e^{-x}}{(1 + e^{-x})} \right)$$

$$= \frac{e^{-x}}{(1 + e^{-x})^2}$$

Therefore,

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

Error Gradient for a Sigmoid Unit

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_d (t_d - o_d) \left(-\frac{\partial o_d}{\partial w_i} \right) \\ &= - \sum_d (t_d - o_d) \frac{\partial o_d}{\partial net_i} \frac{\partial net_i}{\partial w_i}\end{aligned}$$

But we know:

$$\begin{aligned}\frac{\partial o_d}{\partial net_i} &= \frac{\partial (net_d)}{\partial net_d} = o_d(1 - o_d) \\ \frac{\partial net_i}{\partial w_i} &= \frac{\partial (\vec{w} \cdot \vec{x}_d)}{\partial w_i} = x_{i,d}\end{aligned}$$

So:

$$\frac{\partial E}{\partial w_i} = - \sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_{i,d}$$

Error terms (δ) in Perceptron and Delta Rule

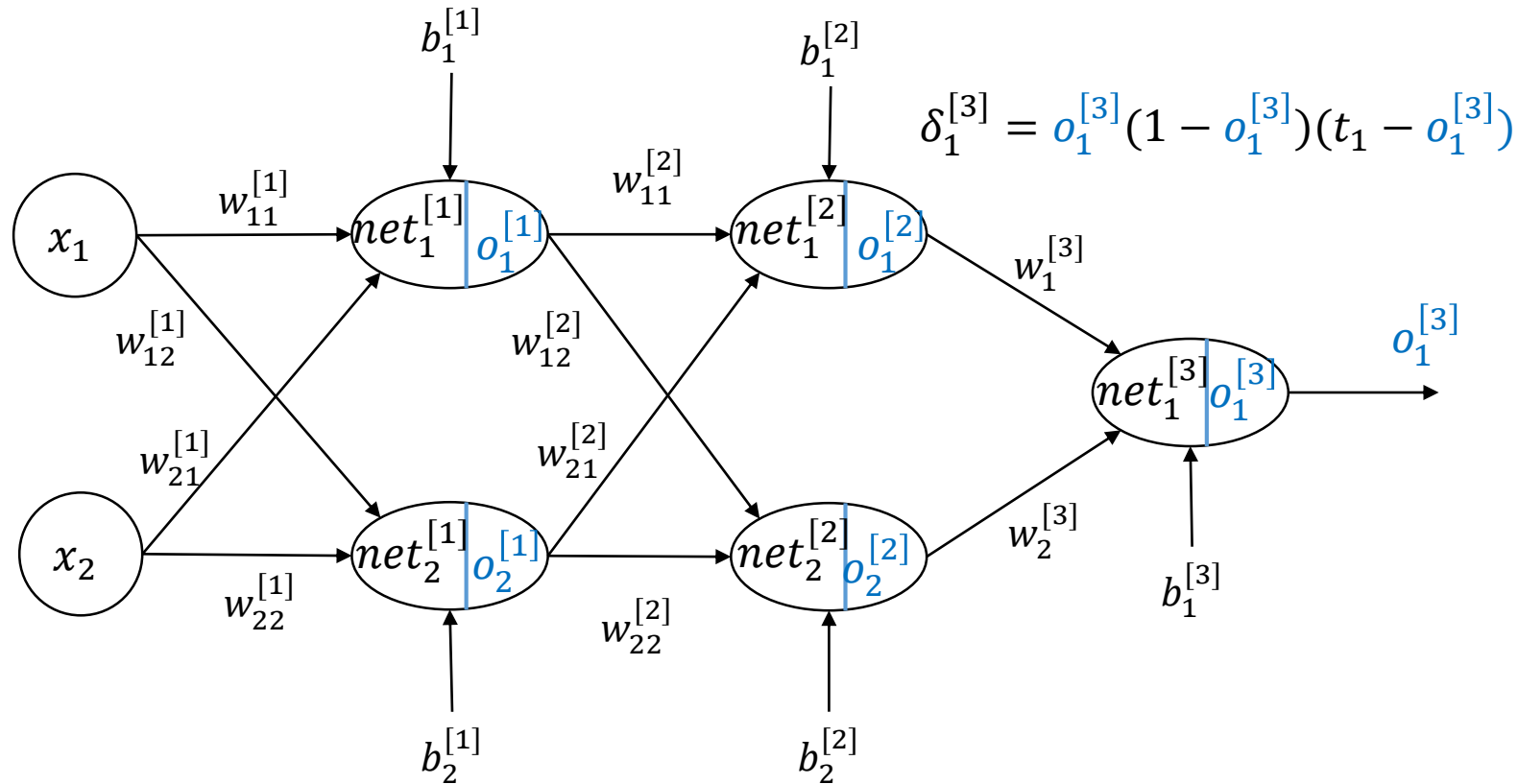
Perceptron Training rule

- $w_i = w_i + \Delta w_i$
- $\Delta w_i = \eta(t - o)x_i$
- $\Delta w_i = \eta(\delta)x_i$

Delta Rule

- $w_i = w_i + \Delta w_i$
- $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$
- $\Delta w_i = \eta \{ \sum_{d \in D} (t_d - o_d) \} x_{i,d}$
- $\Delta w_i = \eta(\delta)x_{i,d}$

forward propagation



$$w^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} \end{bmatrix}$$

$$w^{[2]} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} \end{bmatrix}$$

$$b^{[2]} = \begin{bmatrix} b_1^{[2]} \\ b_2^{[2]} \end{bmatrix}$$

$$w^{[3]} = \begin{bmatrix} w_{11}^{[3]} & w_{12}^{[3]} \\ w_{21}^{[3]} & w_{22}^{[3]} \end{bmatrix}$$

$$b^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \end{bmatrix}$$

$$b^{[3]} = [b_1^{[3]}]$$

Backpropagation Neural Network

Initialize all network weights to small random numbers(e.g., between -0.05 and 0.05)

Until the termination condition is met, Do

For each training example $(\vec{x}, \vec{t})$, Do

%Feed forward the input

Input the instance $\vec{x}$ to the network and compute the output o , of every unit in the network.

%Back propagate the error

For each network output unit k in layer L , i.e., $a_k^{(L)}$, calculate its error term $\delta_k^{(L)}$

$$\delta_k^{(L)} = o_k^{(L)} (1 - o_k^{(L)}) (t_k - o_k^{(L)})$$

For layers l in $L-1$ to 1

For each hidden unit h in layer l , i.e., $o_h^{(l)}$, calculate its error term $\delta_h^{(l)}$

$$\delta_h^{(l)} = o_h^{(l)} (1 - o_h^{(l)}) \sum_{a_k^{l+1} \in \text{Downstream}(a_h^l)} w_{kh}^{(l+1)} \delta_k^{(l+1)}$$

Update each weight in each layer l

$$w_{ji}^{(l)} = w_{ji}^{(l)} + \Delta w_{ji}^{(l)}$$

$$\Delta w_{ji}^{(l)} = \eta \delta_j^{(l)} o_i^{(l-1)}$$

Multi-layer NN Error

- Redefining E as there are more than one output units

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2$$

t_{kd} and o_{kd} are the target and output values associated with the k^{th} output unit for training example d

Useful for *batch gradient descent*

However, for *stochastic GD*, the error is calculated for a single training example d and weights are updated to reduce the error E_d w.r.t the single example

$$E_d(\vec{w}) = \frac{1}{2} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2$$

$$\begin{aligned} w_i &= w_i + \Delta w_i \\ \Delta w_i &= -\eta \frac{\partial E}{\partial w_i} \end{aligned}$$

Weight Update Rule

- The Backprop weight update rule is based on the gradient descent method:
 - It takes a step in the direction yielding the maximum decrease of the network error E .
 - This direction is the opposite of the gradient of E .
- Iteration of the Backprop algorithm is usually terminated when the sum of squares of errors of the output values for all training data in an epoch is less than some threshold such as 0.01

$$w_{ij}^{new} = w_{ij}^{old} + \Delta w_{ij} \qquad \Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Update weights and biases

➤ **Epoch:**

- One iteration through the entire training set is called an epoch.

➤ **Epoch updating:**

- Weights and biases are accumulated and updated only after all training samples are processed.

➤ **Mini-batch updating:**

- This method divides the training set into small batches. The model updates its weights and biases after each batch is processed.

Update weights and biases

When to use each method:

➤ **Case Updating (SGD):**

- Best for large datasets. It updates weights immediately after each training example, which can lead to faster learning and better performance on new data.

➤ **Epoch Updating:**

- Ideal for smaller datasets or limited computational resources. It updates weights after processing the entire dataset, which can be more memory-efficient.

➤ **Mini-Batch Updating:**

- Commonly used because it strikes a balance between efficiency and quick updates. It processes a small batch of data at a time, benefiting from both memory efficiency and faster convergence.

Backprop learning algorithm

n=1;

initialize **weights** randomly;

while (stopping criterion not satisfied or $n < \text{max_iterations}$)

for each example $(\mathbf{x}, d)$

- run the network with input $\mathbf{x}$ and compute the output y
- update the weights in backward order starting from those of the output layer:

 with computed using the (generalized) Delta rule

end-for

$n = n+1$;

end-while;

Steps in Back propagation Algorithm

➤ Initialize the weights and biases.

The weights in the network are initialized to random numbers from the interval $[-0.5, 0.5]$.

Each unit has a **bias** associated with it

The biases are similarly initialized to random numbers from the interval $[-1, 1]$.

➤ Feed the training sample.

➤ Propagate the inputs forward; we compute the net input and output of each unit in the hidden and output layers.

➤ back propagate the error.

➤ Update weights and biases to reflect the propagated errors.

➤ Terminating conditions.

Stopping criteria

- Total mean squared error change:
 - Back-prop is considered to have converged when the absolute rate of change in the average squared error per epoch is sufficiently small (in the range $[0.1, 0.01]$).
- Generalization based criterion:
 - After each epoch, the NN is tested for generalization.
 - If the generalization performance is adequate then stop.
 - If this stopping criterion is used then the part of the training set used for testing the network generalization will not be used for updating the weights.

Constructive vs Destructive Algorithm

- **Constructive** algorithms take a **minimal** network and **build up** new layers nodes and connections during training.
- **Destructive** algorithms take a **maximal** network and **prunes** unnecessary layers nodes and connections during training.

Network Pruning and Rule Extraction

Network pruning

Fully connected network will be hard to articulate

N input nodes, h hidden nodes and m output nodes lead to $h(m+N)$ weights

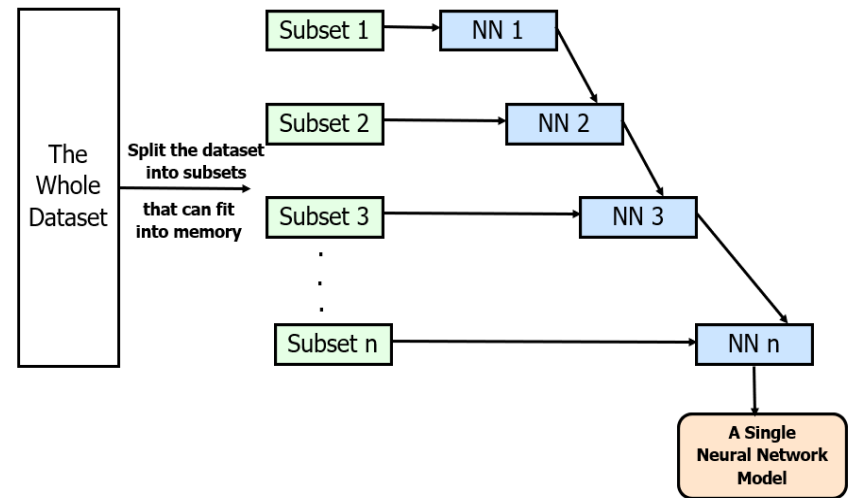
Pruning: Remove some of the links without affecting classification accuracy of the network

Advanced Features of Neural Network

- Training with Subsets
- Modular Neural Network
- Evolution of Neural Network

Training with Subsets

- Select subsets of data
- Build new classifier on subset
- Aggregate with previous classifiers
- Compare error after adding classifier
- Repeat as long as error decreases



Modular Neural Network

Made up of a combination of several neural networks.

The idea is to reduce the load for each neural network as opposed to trying to solve the problem on a single neural network.

Evolving Network Architectures

- **Small networks** without a hidden layer can't solve problems such as XOR, that are **not linearly separable**.
- **Large networks** can easily **overfit** a problem to match the training data, **limiting** their ability to **generalize** a problem set.

NN DESIGN ISSUES

- Network Topology
- Data representation
- Network Parameters
- Training
- Validation

Network Topology

- The number of layers and neurons depend on the specific task.
- In practice this issue is solved by trial and error.
- Two types of adaptive algorithms can be used:
 - start from a large network and successively remove some neurons and links until network performance degrades.
 - begin with a small network and introduce new neurons until performance is satisfactory.

Network parameters

- How are the weights initialized?
- How is the learning rate chosen?
- How many hidden layers and how many neurons?
- How many examples in the training set?

Initialization of weights

- In general, initial weights are randomly chosen, with typical values between -0.5 and 0.5.
- If some inputs are much larger than others, random initialization may bias the network to give much more importance to larger inputs.
- In such a case, weights can be initialized as follows:

$$w_{ij} = \pm \frac{1}{2N} \sum_{i=1, \dots, N} \frac{1}{|x_i|}$$

For weights from the input to the first layer

$$w_{jk} = \pm \frac{1}{2N} \sum_{i=1, \dots, N} \frac{1}{\phi(\sum w_{ij} x_i)}$$

For weights from the first to the second layer

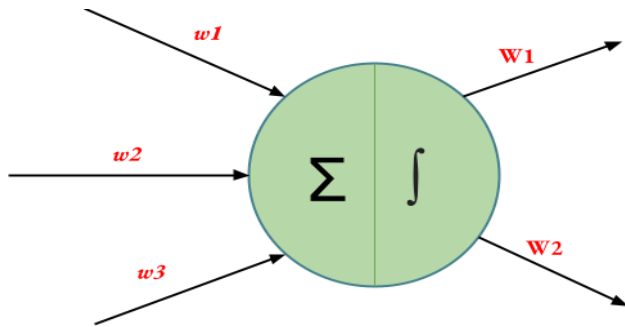
Weight Initialization Techniques

- **Random Initialization**

Random initialization assigns random values except for zeros as weights to neuron paths. Problems such as Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem might occur.

Random Normal: The weights are initialized from values in a normal distribution.

$$w_i \sim N(0, 1)$$



For the neuron,

fan_in = 3 (Number of input paths towards the neuron)

fan_out = 2 (Number of output paths towards the neuron)

- **Xavier Normal/Glorot Initialization**

The weights are assigned from values of a normal distribution as follows:

$$w_i \sim N(0, \sigma)$$

Here, σ is given by:

$$\sigma = \sqrt{\frac{2}{fan_{in} + fan_{out}}}$$

Weight Initialization Techniques

- **Xavier Uniform/Glorot Initialization**

The weights are assigned from values of a uniform distribution as follows:

$$w_i \sim U\left(-\sqrt{\frac{6}{fan_{in} + fan_{out}}}, \sqrt{\frac{6}{fan_{in} + fan_{out}}}\right)$$

- **He Normal Initialization**

The weights are assigned from values of a normal distribution as follows:

$$w_i \sim N(0, \sigma)$$

Here, σ is given by:

$$\sigma = \sqrt{\frac{2}{fan_{in}}}$$

- **He Uniform Initialization**

The weights are assigned from values of a uniform distribution as follows:

$$w_i \sim U\left(-\sqrt{\frac{6}{fan_{in}}}, \sqrt{\frac{6}{fan_{in}}}\right)$$

He Uniform Initialization is suitable for layers where **ReLU** activation function is used

Choice of learning rate

- The right value of η depends on the application.
- Values between 0.1 and 0.9 have been used in many applications.
- Other heuristics is that adapt η during the training as described in previous slides.

Other terminologies

Momentum factor

used for convergence when momentum factor is added to weight update process.

Vigilance parameter (ρ)

Determines how similar new input data must be to existing categories to be grouped together. It typically ranges from 0 to 1:

High ρ

The network is very selective, creating many small, specific categories.

Low ρ

The network is less selective, forming fewer, broader categories.
This parameter helps balance the network's ability to recognize new patterns and generalize from existing ones.

Faster Convergence

- Backpropagation may requires many *epochs* to converge
- Some ideas to overcome this
 - *Stochastic learning*
 - **Update weights after each training example**
 - *Momentum*
 - **Add fraction of previous update to current update**
 - **Faster convergence**

Training

Rule of thumb:

the number of training examples should be at least five to ten times the number of weights of the network.

Other rule:

$$N > \frac{|W|}{(1-a)}$$

|W|= number of weights
a=expected accuracy on test set

Learning Rules in ANN

- Learning rule enhances the ANN's performance by applying this rule over the network.
- Thus learning rule updates the weights and bias levels of a network when certain conditions are met in the training process.

Types-

- Hebbian Learning Rule
- Perceptron Learning Rule
- Delta Learning Rule
- Widrow-Hoff learning Rule
- Winner-Take-All learning rules

Hebbian Learning Rule

FEED FORWARD UNSUPERVISED LEARNING

- The learning signal is equal to the neuron's output

$$r \triangleq f(\mathbf{w}_i^t \mathbf{x}) \quad \Delta w_i = \eta f(\mathbf{w}_i^t \mathbf{x}) \mathbf{x}$$

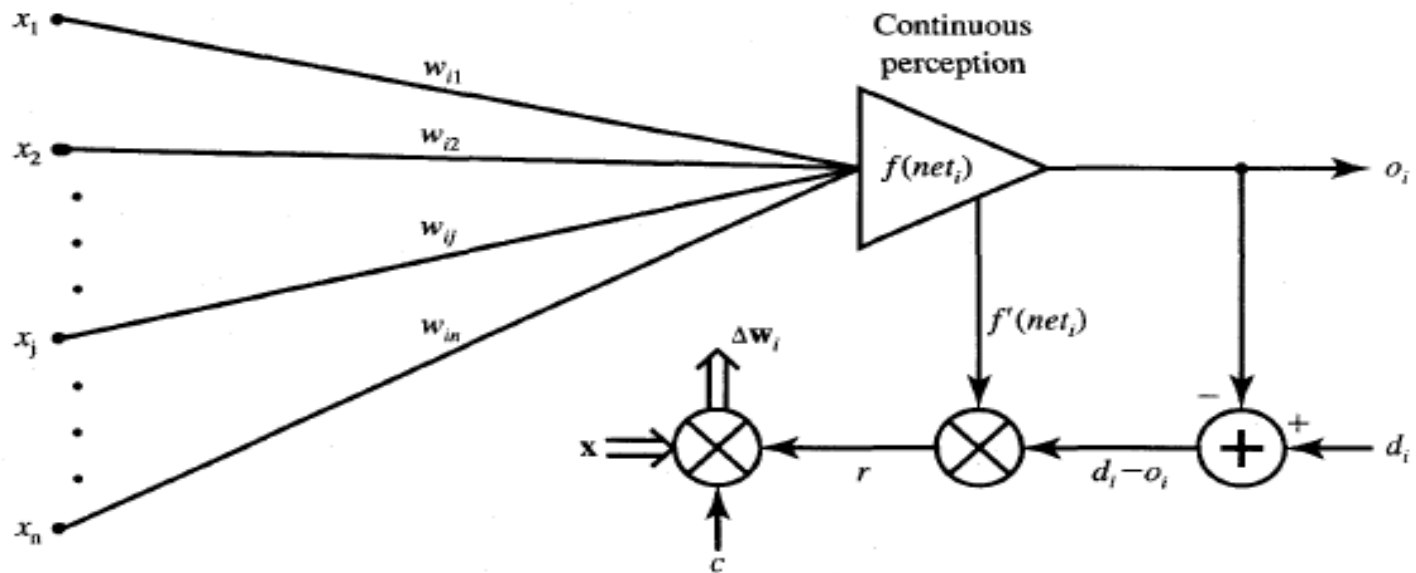
This can be written briefly as

$$\Delta w_{ij} = \eta o_i x_j, \quad \text{for } j = 1, 2, \dots, n$$

Hebbian Learning strengthens connections between neurons that fire together and weakens those that do not. If there is no correlation, weights remain unchanged. It helps the network learn patterns effectively.

Delta Learning Rule

- Only valid for continuous activation function
- Used in supervised training mode
- It is also known as the Least Mean Square method and it minimizes error over all the training patterns.
- It is based on a gradient descent approach which continues forever. It states that the modification in the weight of a node is equal to the product of the error and the input where the error is the difference between desired and actual output.



Delta Learning Rule Contd.

$$r \triangleq [d_i - f(\mathbf{w}_i^t \mathbf{x})] f'(\mathbf{w}_i^t \mathbf{x})$$

Learning rule is derived from the condition of least squared error.

Calculating the gradient vector with respect to w_i

$$E \triangleq \frac{1}{2} (d_i - o_i)^2$$

$$\nabla E = -(d_i - o_i) f'(\mathbf{w}_i^t \mathbf{x}) \mathbf{x}$$

The components of the gradient vector are

$$\frac{\partial E}{\partial w_{ij}} = -(d_i - o_i) f'(\mathbf{w}_i^t \mathbf{x}) x_j, \text{ for } j=1,2,\dots, n$$

Minimization of error requires the weight changes to be in the negative gradient direction

$$\begin{aligned} \Delta w_i &= -\eta \nabla E \\ \Delta w_i &= \eta (d_i - o_i) f'(\mathbf{w}_i^t \mathbf{x}) \mathbf{x} \end{aligned}$$

Widrow-Hoff learning Rule

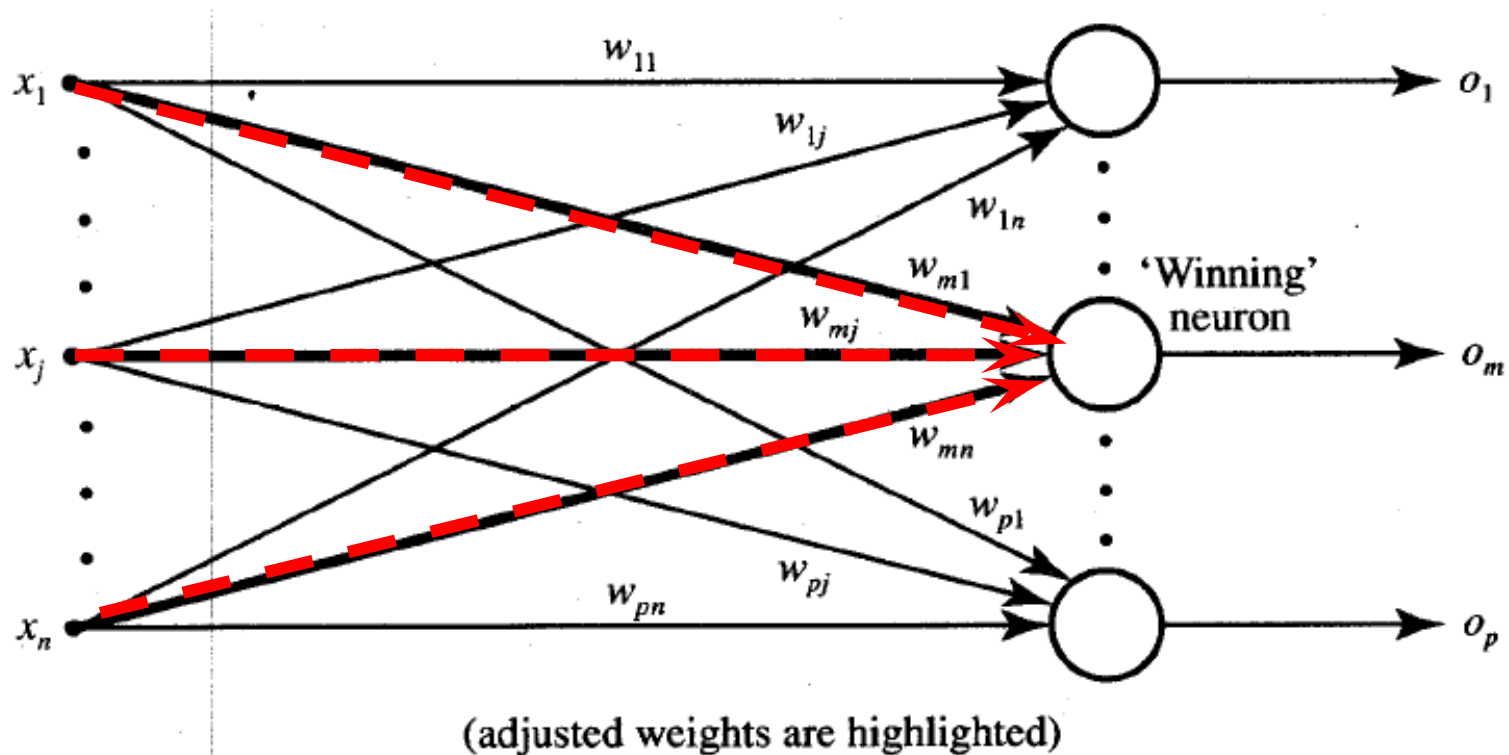
- Also called as least mean square learning rule
- Introduced by Widrow(1962), used in supervised learning
- Independent of the activation function
- Special case of delta learning rule wherein activation function is an identity function ie $f(net)=net$
- Minimizes the squared error between the desired output value d_i and net_i

$$r \triangleq [d_i - (\mathbf{w}_i^t \mathbf{x})]$$

The weight vector increment under this learning rule is

$$\Delta \mathbf{w}_i = \eta (d_i - \mathbf{w}_i^t \mathbf{x}) \mathbf{x}$$

Winner-Take-All learning rules



Winner-Take-All Learning rule Contd...

- Can be explained for a layer of neurons
- Example of competitive learning and used for unsupervised network training
- Learning is based on the premise that one of the neurons in the layer has a maximum response due to the input x
- This neuron is declared the winner with a weight

$$w_m = [w_{m1} \quad w_{m2} \quad \dots \quad w_{m3}]^t$$

Its increment is computed as follows

$$\Delta w_m = \eta(x - w_m)$$

The Winner Selection is based on the following criterion of maximum activation among All p neurons participating in a competition

$$w_m^t x = \max_{(i=1,2,\dots,p)} (w_i^t) x$$

THANK YOU